MBLP131 · İşletim Sistemleri Temelleri · Öğretim üyesi kopyası

Final Sınavı (Yarıyıl Sonu) Örnek Kâğıdı --- Cevap Anahtarı

Çözümler ve puanlama ölçütleri. Her sayı `code/theory_check.py` ile hesaplandı; her komut yeni oluşturulmuş bir `~/lab` içinde çalıştırıldı (`code/answers_sample.sh`).

Genel puanlama kuralları. Yöntem puanı verin: doğru formülle yapılan bir aritmetik hata en çok bir puan götürür. Farklı ama doğru bir komut, seçenek sırası ya da boru hattı tam puan alır (yaygın seçenekler aşağıda listelendi). Farklı ama tutarlı bir uzlaşım kullanan (ör. sadeleştirilmiş TLB formülü) ve bunu belirten öğrenciler tam puan alır. [Puan ağırlıklarını gerçek final kâğıdına göre uyarlayın.]

Bölüm A (20 puan: her biri 2, kısmi puan yok)

A1	A2	A3	A4	A5	A6	A7	A8	A9	A10
b	c	d	c	b	b	b	a	b	c

Notlar: A3 — PCB durum, yazmaçlar, zamanlama, bellek ve dosya bilgilerini tutar; program metnini tutmaz. A8 — $7 = rwx$, $5 = r{-}x$, $4 = r{-}{-}$.

Bölüm B (12 puan)

B1. Program C kütüphanesindeki `write()` API fonksiyonunu çağırır; ince sarmalayıcı sistem çağrısı numarasını (x86-64 Linux'ta `rax` yazmacı) ve parametreleri (yazmaçlar) yerleştirir ve tuzak komutunu (`syscall`) çalıştırır. İşlemci mod bitini çekirdek moduna (0) çevirir, dönüş adresini kaydeder ve çekirdeğin tek sistem çağrısı giriş noktasına atlar; çekirdek numarayı sistem çağrısı tablosunda indeks olarak kullanır ve işleyiciyi (`sys_write`) çalıştırır. Dönüşte sonuç bir yazmaca konur, mod biti yeniden kullanıcı moduna (1) çekilir ve yürütme tuzaktan sonraki komuttan devam eder.

Puanlama. API/sarmalayıcı 1; numara ve tablo 1; tuzak ve mod geçişi 1; sonuçla kullanıcı moduna dönüş 1.

B2. Yarış durumu, birden çok iş parçacığı ortak veriye eşzamanlı eriştiğinde, en az biri yazdığında ve sonuç zamanlamaya bağlı olduğunda ortaya çıkar. `counter++` üç adımdır (yazmaca yükle, topla, sakla); iki iş parçacığı da eski değeri, hiçbiri saklamadan önce yüklerse bir artırma kaybolur. Bir çözüm *karşılıklı dışlamayı*, *ilerlemeyi* ve *sınırlı beklemeyi* güvence altına almalıdır.

Puanlama. tanım 1; counter++'ın yükle–topla–sakla olarak açıklanması ve kaybolan güncelleme 1; üç koşul 2 (her biri 1, en çok 2).

B3. Boşa çalışma: süreçlerin çerçeve sayısı etkin olarak kullandıkları sayfalardan (çalışma kümelerinden) azdır; bu yüzden sürekli sayfa hatası verir, diski beklerler ve işlemci kullanımı çöker; saf bir zamanlayıcı da bunun üzerine daha çok süreç kabul eder. Çalışma kümesi $WS(t, \Delta)$, son Δ başvuruda kullanılan farklı sayfaların kümesidir; işletim sistemi her sürece en az $|WS|$ çerçeve verir ve toplam talep $D = \sum |WS_i|$ çerçeve sayısını aşarsa yeni süreç başlatmak yerine bir süreci askıya alır.

Puanlama. nedeniyle birlikte boşa çalışma tanımı 2; çalışma kümesi tanımı 1; nasıl kullanıldığı (D ile çerçeve sayısı,

askıya alma) 1.

Bölüm C (36 puan)

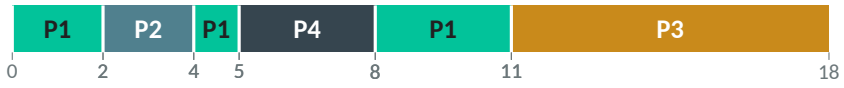
C1 (12). P1 $(0, 6, 3)$, P2 $(2, 2, 1)$, P3 $(3, 7, 4)$, P4 $(5, 3, 2)$; (varış, patlama, öncelik) olarak.

(a) FCFS



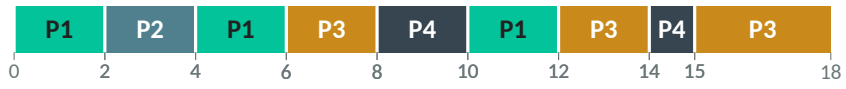
Bekleme $0, 4, 5, 10$: ortalama $19/4 = 4.75$ ms. Dönüş $6, 6, 12, 13$: ortalama $37/4 = 9.25$ ms.

(b) Kesintili Öncelik



P2 (öncelik 1) 2'de P1'i keser; 4'te P1 devam eder; P4 (öncelik 2) onu 5'te keser; P1 11'de biter, P3 en sonda. Bekleme $5, 0, 8, 0$: ortalama $13/4 = 3.25$ ms. Dönüş $11, 2, 15, 3$: ortalama $31/4 = 7.75$ ms.

(c) Round Robin, $q = 2$



$t = 2$'de P2, P1'in zaman dilimi dolduğu anda gelir ve kuyruğa önce girer (kuyruk P2, P1); P3 3'te gelir (kuyruk P1, P3); P4 5'te gelir. Tamamlanma $12, 4, 18, 15$. Bekleme $6, 0, 8, 7$: ortalama $21/4 = 5.25$ ms. Dönüş $12, 2, 15, 10$: ortalama $39/4 = 9.75$ ms.

Puanlama. algoritma başına 4 puan: Gantt şeması 2, ortalama bekleme 1, ortalama dönüş 1. Başka açılardan tutarlı ama yanlış bir RR eşitlik sırası: -1.

C2 (8). (a) 2 KB $= 2^{11} = 2048$: $p = \lfloor 5000/2048 \rfloor = 2$, $d = 5000 - 4096 = 904$; sayfa $2 \to$ çerçeve 6; $a_{\text{fiz}} = 6 \times 2048 + 904 = 13\,192$. (b) $\lfloor 9000/2048 \rfloor = 4$, ama tabloda yalnızca 0–3 sayfaları var: adres geçersizdir, MMU işletim sistemine tuzak üretir (süreç genellikle sonlandırılır: segmentation fault). (c) EAT $= 10 + (2 - 0.8) \times 90 = 10 + 108 = 118$ ns $(= 0.8 \times 100 + 0.2 \times 190 = 80 + 38)$; sadeleştirilmiş uzlaşımla ($t_{\text{TLB}} = 0$): $0.8 \times 90 + 0.2 \times 180 = 108$ ns. (d) İlk uyan: 450 KB (delik 2); en iyi uyan: 300 KB (delik 4); en kötü uyan: 500 KB (delik 5).

Puanlama. (a) p, d birer puan, a_{fiz} 1; (b) 1; (c) 2 (formül 1, değer 1); (d) 2 (üçü de doğru; ikisi doğru: 1).

C3 (8). FIFO 9, OPT 6, LRU 7 hata (kalın = yeni yüklenen sayfa).

FIFO

başv.	**5**	**4**	**1**	**5**	**0**	**5**	**4**	**0**	**2**	**1**	**0**	**1**
çr. 0	**5**	5	5	5	**0**	0	0	0	**2**	2	2	2
çr. 1		**4**	4	4	4	**5**	5	5	5	**1**	1	1
çr. 2			**1**	1	1	1	**4**	4	4	4	**0**	0
hata	**F**	**F**	**F**		**F**	**F**	**F**		**F**	**F**	**F**	

OPT

başv.	**5**	**4**	**1**	**5**	**0**	**5**	**4**	**0**	**2**	**1**	**0**	**1**
çr. 0	**5**	5	5	5	5	5	5	5	**2**	**1**	1	1
çr. 1		**4**	4	4	4	4	4	4	4	4	4	4
çr. 2			**1**	1	**0**	0	0	0	0	0	0	0
hata	**F**	**F**	**F**		**F**				**F**	**F**		

LRU

başv.	**5**	**4**	**1**	**5**	**0**	**5**	**4**	**0**	**2**	**1**	**0**	**1**
çr. 0	**5**	5	5	5	5	5	5	5	**2**	2	2	2
çr. 1		**4**	4	4	**0**	0	0	0	0	0	0	0
çr. 2			**1**	1	1	1	**4**	4	4	**1**	1	1
hata	**F**	**F**	**F**		**F**		**F**		**F**	**F**		

OPT: 0'da 1'i çıkar (en geç yeniden kullanılan); 2'de hem 5 hem 4 bir daha hiç kullanılmaz, 1'de de hem 2 hem 4 bir daha hiç kullanılmaz — aralarındaki her seçim aynı sayıyı verir; bu sütunlarda farklı çerçeve içeriklerini kabul edin. LRU: 0'da 4'ü, 4'te 1'i, 2'de 5'i, 1'de 4'ü çıkar.

Puanlama. FIFO 3, OPT 2, LRU 3 (FIFO ve LRU için tablo 2 + sayı 1; OPT: sayı 1, kararlar 1).

C4 (8). (a) EAT $= 0.9995 \times 100 + 0.0005 \times 5\,000\,000 = 99.95 + 2500 = 2599.95$ ns, RAM'den yaklaşık 26 kat yavaş. (b) Bayt $100 \times 50 = 5000$; blok $\lfloor 5000/512 \rfloor = 9$; konum $5000 - 9 \times 512 = 392$; son bayt 5049, hâlâ blok 9'da ($10 \times 512 = 5120$): sığar. (c) $U(4) = 1 - 0.5^4 = 0.9375$; hızlanma $\leq 1/(0.25 + 0.75/3) = 1/0.5 = 2$; sınır $1/0.25 = 4$.

Puanlama. (a) formül 1, değer 1, kat 1; (b) blok 1, konum 1, sığma 1; (c) 1 + 1.

Bölüm D (14 puan: her biri 2)

Puanlama. 2 = doğru ve eksiksiz; 1 = doğru komut ama yanlış ya da eksik seçenek/argüman; aksi hâlde 0.

Beklenen yanıtlar ve gerçek çıktıları:

D1 `ls -lah docs` (ya da `ls -la -h docs`).

```
student@okan-lab:~/lab$ ls -lah docs
total 36K
drwxrwxr-x  2 student student 4.0K Oct 12 08:50  .
drwxrwxr-x 10 student student 4.0K Oct 12 08:50  ..
-rw-rw-r--  1 student student   21 Oct 12 08:50  .settings
-rw-rw-r--  1 student student   53 Oct 12 08:50 'final report.txt'
-rw-rw-r--  1 student student  302 Oct 12 08:50  linux_quotes.txt
-rw-rw-r--  1 student student   16 Oct 12 08:50  notes_01.txt
-rw-rw-r--  1 student student   16 Oct 12 08:50  notes_02.txt
-rw-rw-r--  1 student student   16 Oct 12 08:50  notes_03.txt
-rw-rw-r--  1 student student  578 Oct 12 08:50  syllabus.txt
```

D2 `mv docs/notes_01.txt docs/meeting_01.txt`

```
student@okan-lab:~/lab$ mv docs/notes_01.txt docs/meeting_01.txt
student@okan-lab:~/lab$ ls docs
'final report.txt'   meeting_01.txt   notes_03.txt
 linux_quotes.txt    notes_02.txt     syllabus.txt
```

D3 `touch report{1..5}.txt` (süslü ayraç genişletmesi; `touch report1.txt … report5.txt` 1 puan alır — "kısa" değil).

```
student@okan-lab:~/lab$ touch report{1..5}.txt
student@okan-lab:~/lab$ ls report*
report1.txt  report2.txt  report3.txt  report4.txt  report5.txt
```

D4 `ln -s logs/access.log latest.log` (göreli hedef, bağlantının bulunduğu dizine göre çözülür).

```
student@okan-lab:~/lab$ ln -s logs/access.log latest.log
student@okan-lab:~/lab$ ls -l latest.log
lrwxrwxrwx 1 student student 15 Oct 12 08:50 latest.log -> logs/access.log
student@okan-lab:~/lab$ head -n 1 latest.log
10.0.1.10 - - [12/Oct/2026:08:00:00 +0300] "GET /about.html HTTP/1.1" 404 442
```

D5 `chmod u+x scripts/backup.sh` (sekizli gösterim sembolik değildir: 1 puan).

```
student@okan-lab:~/lab$ touch scripts/backup.sh
student@okan-lab:~/lab$ chmod u+x scripts/backup.sh
student@okan-lab:~/lab$ ls -l scripts/backup.sh
-rwxrw-r-- 1 student student 0 Oct 12 08:50 scripts/backup.sh
```

D6 `kill -9 4321` (ya da `kill -KILL 4321`, `kill -SIGKILL 4321`). Aşağıdaki kayıtta PID `$!`'den alındı; komutları çalıştıran kabuk bir betikti.

```
student@okan-lab:~/lab$ sleep 900 &
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
23954 Ss+  bash /home/student/w15/answers_sample.sh
24000 S+   sleep 900
24002 R+   ps -o pid,stat,cmd
student@okan-lab:~/lab$ kill -9 $!
student@okan-lab:~/lab$ ps -o pid,stat,cmd
  PID STAT CMD
23954 Ss+  bash /home/student/w15/answers_sample.sh
24009 R+   ps -o pid,stat,cmd
```

D7 `du -sh logs`

```
student@okan-lab:~/lab$ du -sh logs
36K     logs
```

Bölüm E (9 puan: her biri 3)

Puanlama. 3 = doğru; 2 = bir aşama yanlış ya da eksik (ör. `sort` olmadan `uniq -c` ya da `head` yok); 1 = doğru fikir, birkaç hata.

E1 `sort data/cities.txt | uniq -c | sort -rn | head -1`

```
student@okan-lab:~/lab$ sort data/cities.txt | uniq -c | sort -rn | head -1
      6 Antalya
```

E2 `grep -c "GET /login " logs/access.log` (ya da `cut -d' ' -f7 logs/access.log | grep -c '^/login$'`). `/login`'den sonraki boşluk olmazsa kalıp `/login2` gibi sayfaları da sayardı — günlüğümüzde böyle sayfa olmadığından burada aynı sayıyı verir.

```
student@okan-lab:~/lab$ grep -c "GET /login " logs/access.log
22
```

E3 `tail -n +2 data/students.csv | sort -t, -k5 -rn | head -3 | cut -d, -f2,5`

```
student@okan-lab:~/lab$ tail -n +2 data/students.csv | sort -t, -k5 -rn | head -3 | cut -d, -
    f2,5
Onur Dogan,100
Elif Yilmaz,99
Emre Dogan,98
```

`tail -n +2` olmadan da başlık satırı `-rn` ile sona sıralanır (anahtarı sayı değildir); bu yüzden sonuç burada aynıdır: kabul edin.

Bölüm F (9 puan)

```bash
#!/bin/bash
# passfail.sh -- how many students reached a given final-exam grade
# usage: ./passfail.sh THRESHOLD          (run inside ~/lab)
if [ $# -ne 1 ]; then
    echo "usage: $0 THRESHOLD" >&2
    exit 1
fi
pass=0
fail=0
for g in $(tail -n +2 data/students.csv | cut -d, -f5); do
    if [ "$g" -ge "$1" ]; then
        pass=$((pass + 1))
    else
        fail=$((fail + 1))
    fi
done
echo "threshold $1: $pass passed, $fail failed"
```

```
student@okan-lab:~/lab$ chmod +x scripts/passfail.sh
student@okan-lab:~/lab$ ./scripts/passfail.sh 50
threshold 50: 30 passed, 10 failed
student@okan-lab:~/lab$ ./scripts/passfail.sh 90
threshold 90: 7 passed, 33 failed
student@okan-lab:~/lab$ ./scripts/passfail.sh
usage: ./scripts/passfail.sh THRESHOLD
student@okan-lab:~/lab$ echo $?
1
```

Puanlama. shebang ve standart hataya kullanım iletisi ile `exit 1` içeren argüman denetimi: 2; final notlarının çıkarılması (`tail -n +2`, `cut -d, -f5` ya da bir `while IFS =, read` döngüsü): 2; sayısal sınamalı döngü (`[ ]` içinde `>` = değil, `-ge`): 2; `$(( ))` ile iki sayaç: 2; istenen biçimde çıktı: 1. Doğruysa bir awk çözümü de kabul edilir.